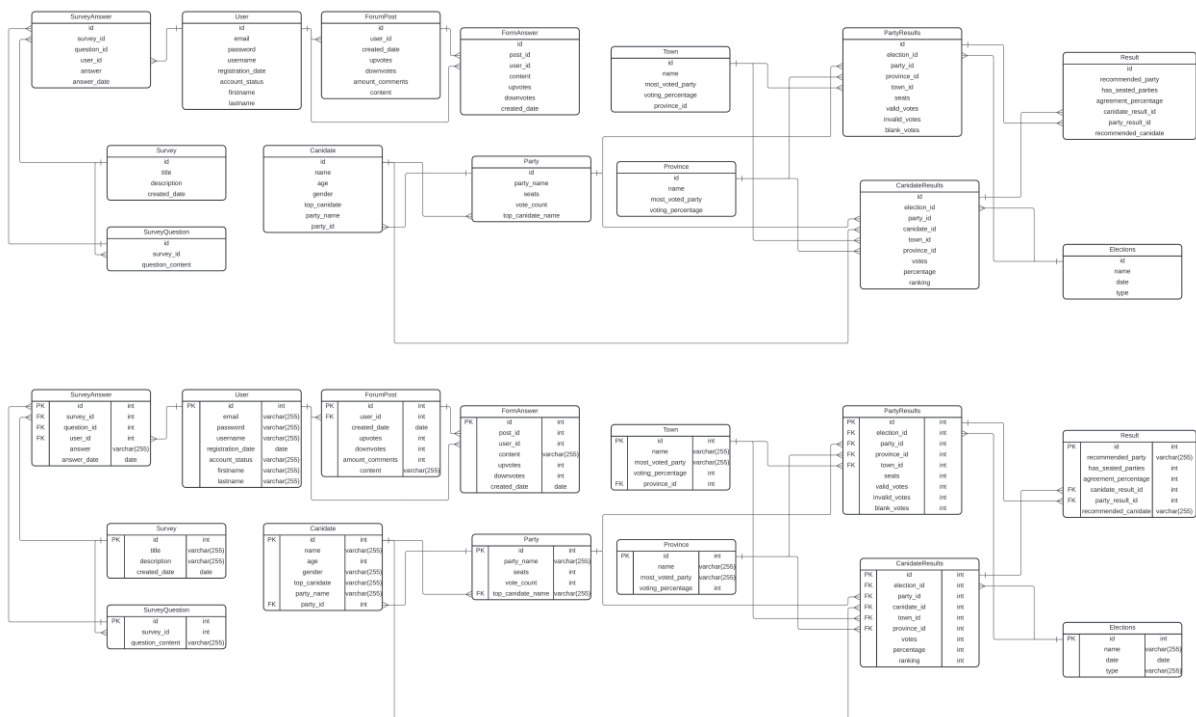


Sprint 3 increment – backend/frontend development:

1. I try to commit daily and commit 3 times a day



2. I created a erd/eerd and changed them based on feedback



Erd: <https://gitlab.fdmci.hva.nl/semester-3-hbo-ict/onderwijs/student-projecten/2025-2026/out-s-t-se/suuzaameebaa24/-/blob/main/docs/Diagrams/erd%20diagram.png>

Eerd: [https://gitlab.fdmci.hva.nl/semester-3-hbo-ict/onderwijs/student-projecten/2025-2026/out-s-t-se/suuzaameebaa24/-/blob/main/docs/Diagrams/e\(e\)rd%20diagram.png](https://gitlab.fdmci.hva.nl/semester-3-hbo-ict/onderwijs/student-projecten/2025-2026/out-s-t-se/suuzaameebaa24/-/blob/main/docs/Diagrams/e(e)rd%20diagram.png)

3. Updated the dod based on given feedback

Dod: <https://gitlab.fdmci.hva.nl/semester-3-hbo-ict/onderwijs/student-projecten/2025-2026/out-s-t-se/suuzaameebaa24/-/blob/main/docs/Afspraken/DoD.md>

4. How I coded

1. The frontend

The very first thing I did was building my own branch and after that I started coding.

For this part I worked with vue and in vue I build a simple form that the admin could fill in so new questions could be added.

The first thing I did was looking up how to work with vue, because before this I never really heard of it.

The first step for building my page was just using basic html and css, after that I started to use functions to for example show messages if for example a input field was not used.

But I also build some other smaller functions like for example if all the input fields are filled then show a modal to confirm you want to send it.

The image displays two screenshots of the KiesOpFeiten web application. The top screenshot shows the 'Nieuwe Vraag Toevoegen' (Add New Question) form. It features a title input field with a warning icon and the text 'Titel is verplicht'. Below this is a text input field for the question title, followed by a section for 'Antwoordopties' (Answer Options) with a text input field and a 'Verwijderen' (Remove) button. At the bottom of the form are two buttons: 'Vraag antwoordoptie toe' (Add answer option) and 'Vraag opslaan' (Save question). The bottom screenshot shows a confirmation modal with the text 'Weet je zeker dat je deze vraag wilt opslaan?' (Are you sure you want to save this question?). It contains two buttons: 'Opslaan' (Save) and 'Annuleren' (Cancel). Below the modal, the 'Vraag opslaan' button is visible again. The footer of the application includes the text 'Waarom KiesOpFeiten?' and a note about social media and news sites.

That modal function was for example build with v-if so if the requirements are met then the modal shows.

Sites like w3schools really helped me with building the functions because vue works a bit differently then what I am used to doing.

2. The backend

This part took a very long time to build for my issue I basically had to send the data from the form to the database which sounded pretty easy because I did this last year, but because of the use of java it was a pretty big learning curve.

Eventually I did manage to send the form data to the database and I did it like this:

First I started by adding a POST request to the Vue and adding what content I want to send. I also had to add to which api I wanted to send it. I also added some messages if something goes wrong then you see a little pop up that the data was not send.

```
try {
  // Send a POST request to your backend to save the survey
  const response = await fetch( input: "http://localhost:8080/api/survey", init: {
    method: "POST",
    headers: { "Content-Type": "application/json" }, // Tell backend we send JSON
    body: JSON.stringify( value: {
      title: title.value,
      description: "Standaard beschrijving", // Just a default description for now (need to discuss if
      questions: questions // The list of questions we built above
    })
  });

  // Check if the server responded with an error
  if (!response.ok) {
    triggerAlert("✖ Serverfout bij het opslaan van de vraag");
  }

  // Show success message to the user
  triggerAlert("✅ Vraag succesvol opgeslagen!");

  // Clear all fields so the form is ready for a new question
  title.value = "";
  answerOptions.value = [""];
}
// If something goes wrong (like backend not running), show an error
catch (err: any) {
```

After that, I started by creating the model files. Models are basically the structure of how the database table is going to look for example, what columns there are and what kind of data is stored in each one, like the id, title, and description. I based my models on my ERD diagram, so they match the relationships I had already planned out there.

```
1 package com.sop.backend.models;
2
3 import jakarta.persistence.*;
4
5 // This entity represents a single question inside a survey.
6 // Each question belongs to one survey.
7
8 @Entity 2 usages  Arman Kokor
9 @Table(name = "SurveyQuestion")
10 public class SurveyQuestion {
11
12     @Id 1 usage
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     private String questionContent; 2 usages
17
18     @ManyToOne 2 usages
19     @JoinColumn(name = "survey_id", nullable = false) // connect to the survey it belongs to
20     private Survey survey;
21
22     // Getters and setters
23     public Long getId() { return id; }  Arman Kokor
24     public String getQuestionContent() { return questionContent; } no usages  Arman Kokor
25     public void setQuestionContent(String questionContent) { this.questionContent = questionContent; } no usages
26     public Survey getSurvey() { return survey; }  Arman Kokor
27     public void setSurvey(Survey survey) { this.survey = survey; }  Arman Kokor
28 }
29
```

Then I created the dto files (Data Transfer Objects). These are used to send and receive clean data between the frontend and backend. They help make sure that the frontend doesn't directly touch the database, which makes everything safer and easier to manage.

```
1 package com.sop.backend.dto;
2
3 import java.time.LocalDate;
4 import java.util.List;
5
6 // This class is used to send and receive survey data.
7 public class SurveyDTO { 1 usage  Arman Kokor
8     private Long id; 2 usages
9     private String title; 2 usages
10     private String description; 2 usages
11     private LocalDate createdDate; 2 usages
12     private List<SurveyQuestionDTO> questions; // list of connected questions 2 usages
13
14     // Below are all the getters and setters
15     // These are needed to read and change data
16     public Long getId() { return id; }  Arman Kokor
17     public void setId(Long id) { this.id = id; }  Arman Kokor
18
19     public String getTitle() { return title; } no usages  Arman Kokor
20     public void setTitle(String title) { this.title = title; } no usages  Arman Kokor
21
22     public String getDescription() { return description; } no usages  Arman Kokor
23     public void setDescription(String description) { this.description = description; } no usages  Arman Kokor
24
25     public LocalDate getCreatedDate() { return createdDate; } no usages  Arman Kokor
26     public void setCreatedDate(LocalDate createdDate) { this.createdDate = createdDate; } no usages  Arman Kokor
27
28     public List<SurveyQuestionDTO> getQuestions() { return questions; } no usages  Arman Kokor
29     public void setQuestions(List<SurveyQuestionDTO> questions) { this.questions = questions; } no usages  Arman Kokor
30 }
31
```

After that, I built the controller. The controller is like the middleman between Vue and the database. It receives the POST request from the frontend, takes the data from the DTO, creates a new survey and questions, and then saves everything using the repositories.

The repositories are what actually talk to the database. They make it easy to save or get data without writing long SQL queries. For example, with one line like `surveyRepository.save(survey)`, I can store a whole survey in the database.

Once everything was saved, the controller sends a response back to Vue, so Vue knows everything worked. I also tested it by checking my console and database to make sure the data was actually stored.

This whole process helped me understand a lot more about how the frontend and backend communicate. Vue sends data to the controller, the controller saves it using the models and repositories, and then sends a clean response back.

